

SUMÁRIO

# Análise dos Instrumentos Urbanísticos Municipais (item 3.2 da tese).....	2
## O que cada script faz	2
## Passo a passo para rodar	3
1) Instale o Python 3.10+ e as dependências:	3
2) Organize os arquivos nesta pasta:	3
3) Rode a análise principal:	4
4) Rode o enriquecimento e a montagem da planilha final:	4
5) (Opcional) Reprocessamento pós-parecer:	4
## Erros comuns	4
## Reprodutibilidade	5

ANÁLISE DOS INSTRUMENTOS URBANÍSTICOS MUNICIPAIS (ITEM 3.2 DA TESE)

Código usado na tese de doutorado de Yasmin Viana ("Geoinformação e gestão pública municipal na Baixada Fluminense Histórica") para o item ****3.2 — Análise das Legislações e Instrumentos Municipais****, que mede o ****Grau de Integração Municipal**** da geoinformação em 31 instrumentos urbanísticos (Cadastro Técnico Multifinalitário, Planta Genérica de Valores, Plano Diretor, Plano de Mobilidade, Plano de Saneamento e Plano Diretor de Tecnologia da Informação) dos 8 municípios da Baixada Fluminense Histórica.

O vocabulário, os pesos de força técnica, os temas e a rubrica de classificação usados por este código são os mesmos publicados nos Anexos B e C do `Texto-Tese-Modificado.docx` (ver também o item 3.2.2 da metodologia) e ficam no arquivo `Parametros\_LDO\_LOA.xlsx` — o mesmo arquivo de parâmetros usado na análise das LDOs/LOAs (pasta `../3.3\_LDO\_LOA/`), pois os dois eixos compartilham parte do vocabulário técnico. Copie esse arquivo para esta pasta antes de rodar (veja o passo 1 abaixo).

O QUE CADA SCRIPT FAZ

O pipeline tem ****quatro etapas em sequência**** (rode nesta ordem):

1. ****`analise\_instrumentos.py`**** — script principal. Lê os PDFs dos instrumentos urbanísticos, extrai o texto (com OCR via Tesseract nas páginas digitalizadas, usando o cache em `ocr\_cache.py`), busca o vocabulário técnico, mede a ancoragem territorial pelo contexto, calcula peso = força × ancoragem, classifica a institucionalização (Explícita / Implícita / Prática / Ausente) e detecta menções a leis federais. Gera `resultados.json` e `ocorrencias.json` na pasta de saída indicada.
2. ****`enhance.py`**** — pacote de robustez: reprocessa os PDFs para datar cada instrumento (ano da norma, ano da capa), determinar a vigência jurídica e a modalidade, aplicando o corte temporal de 2015 usado na tese. Lê `resultados.json`/`ocorrencias.json` da pasta atual e gera `analise\_final.json`.
3. ****`build\_final.py`**** — monta a planilha Excel final (`Analise\_Institucionalizacao\_Geoinformacao.xlsx`), com abas por instrumento, por município, gráficos e formatação condicional, a partir de `analise\_final.json` e `ocorrencias.json`.

4. **reprocess.py** *(opcional — reprocessamento pós-parecer da banca)* — aplica exclusões de documentos contaminados do corpus, expressões de falso-positivo e de "cadastro fiscal fraco", deduplica menções repetidas entre documentos e recalcula o Grau de Integração apenas com ocorrências aproveitáveis em instrumentos vigentes. Gera `resultados_v2.json` e `ocorrencias_v2.json`.

> **Atenção:** este script depende de um arquivo `anexo_params.json` (com as listas de

> expressões de falso-positivo e de cadastro fiscal fraco) que não foi localizado nas pastas de

> trabalho originais durante a organização deste pacote. Se você for reaplicar o

> reprocessamento, será preciso primeiro exportar esse arquivo a partir das abas de apoio do

> `Parametros_LDO_LOA.xlsx` (seção "apoio" com `EXPRESSOES_FALSO_POSITIVO` e

> `EXPRESSOES_CADASTRO_FISCAL_FRACO`) ou reconstruí-lo manualmente antes de rodar este passo. Os

> passos 1 a 3 acima funcionam de forma independente, sem precisar deste arquivo.

`ocr_cache.py` não é chamado diretamente — é um utilitário de OCR "resumível" (salva o progresso

por página, permite continuar de onde parou) usado internamente. `fluxograma.png` é um diagrama

ilustrando visualmente as etapas acima.

PASSO A PASSO PARA RODAR

****1) Instale o Python 3.10+ e as dependências:****

```
```bash
pip install -r requirements.txt
```
```

Você também precisa do **Tesseract OCR** instalado no sistema, com o pacote de idioma português

(`por`). No Windows, use o instalador da UB Mannheim e marque "Portuguese" em "Additional language

data"; no Linux, `sudo apt install tesseract-ocr tesseract-ocr-por`; no Mac, `brew install tesseract tesseract-lang`.

****2) Organize os arquivos nesta pasta:****

...

```
3.2_Instrumentos_Urbanisticos/
  analyse_instrumentos.py
  enhance.py
  build_final.py
```

```

reprocess.py      (opcional)
ocr_cache.py
requirements.txt
Parametros_LDO_LOA.xlsx  <- copie de ../3.3_LDO_LOA/
Instrumentos/      <- crie esta pasta com os PDFs (ver estrutura abaixo)
    CTM - Japeri.pdf
    PGV - Belford Roxo.pdf
    Plano Diretor - Duque de Caxias.pdf
    ... (demais PDFs, um arquivo por instrumento e município)
...

```

Os PDFs originais analisados na tese estão em
`RevisaoMetodologia\_Instrumentos Urbanísticos/Instrumentos/` na pasta da tese —
copie-os para a
subpasta `Instrumentos/` aqui antes de rodar (eles NÃO foram incluídos neste pacote de
código por
serem documentos públicos pesados, alguns com mais de 40 MB).

****3) Rode a análise principal:****

```

```bash
mkdir -p saida
python3 analise_instrumentos.py --pasta Instrumentos --parametros
Parametros_LDO_LOA.xlsx --saida saida
cp saida/resultados.json saida/ocorrencias.json .
...

```

(o comando `cp` copia os resultados para a pasta atual, de onde os próximos scripts os  
leem)

### **\*\*4) Rode o enriquecimento e a montagem da planilha final:\*\***

```

```bash
python3 enhance.py
python3 build_final.py
...

```

O resultado final é `Analise\_Institucionalizacao\_Geoinformacao.xlsx`, na pasta atual.

****5) (Opcional) Reprocessamento pós-parecer:****

Só depois de resolver a dependência `anexo\_params.json` (ver observação acima):

```

```bash
python3 reprocess.py
...

```

## **## ERROS COMUNS**

- `ModuleNotFoundError` → repita o passo 1 (`pip install -r requirements.txt`).

- `TesseractNotFoundError` → confirme a instalação do Tesseract e do idioma ``por`` (passo 1).
- Demora muito → normal em documentos digitalizados grandes (alguns PDFs de Plano de Mobilidade e Plano de Saneamento têm mais de 40 MB e dezenas de páginas escaneadas); o cache em ``ocr_cache.py`` permite retomar sem reprocessar páginas já lidas.
- `FileNotFoundError: resultados.json` ao rodar ``enhance.py`` → confirme que você copiou ``resultados.json`` e ``ocorrencias.json`` da pasta ``saida/`` para a pasta atual (passo 3).

## **## REPRODUTIBILIDADE**

Vocabulário, pesos e rubrica idênticos aos Anexos B e C do ``Texto-Tese-Modificado.docx``. Os resultados publicados na tese (item 4.3) foram gerados com Python 3.12, Tesseract 5 (idioma ``por``) e as versões mínimas de bibliotecas listadas em ``requirements.txt``.